



# ClinicalBridge

Bridging the Clinical Context Gap with an LLM-Powered Multi-Agent System

COP-3442 Prompt Engineering - Spring 2026

| Student            | Student number |
|--------------------|----------------|
| Erkan Işık Bacak   | 2200914        |
| Raymond Lasses     | 2200274        |
| Ata Uzun           | 2103247        |
| Kutlay Başar Aklan | 2202139        |

**Safety notice.** This report describes an educational prototype using entirely simulated data. ClinicalBridge is not a diagnostic tool and must not be used for real clinical decisions.

Final report generated 2026-06-20

# Contents

## Abstract

1. Problem and motivation
2. Objective and scope
3. Requirements interpretation
4. System architecture
5. OpenAI and LangChain implementation
6. Retrieval-augmented generation
7. Memory and tools
8. Simulated dataset
9. Prompt engineering
10. Safety design
11. Evaluation
12. Interfaces and demonstration
13. Limitations
14. Reflection
15. Conclusion
16. References

## Abstract

ClinicalBridge is an educational multi-agent prototype that synthesizes simulated Electronic Health Record, Remote Patient Monitoring, and anamnesis data into a short Clinical Context Brief. The project addresses the clinical context gap: alerts often arrive as isolated measurements while longitudinal history and patient-reported explanations remain in separate systems. Four specialized agents perform triage, EHR retrieval, anamnesis interpretation, and synthesis, while a LangGraph orchestrator coordinates parallel context gathering, critical-alert bypass, audit logging, and final safety validation.

The system uses OpenAI structured outputs through LangChain, OpenAI embeddings with persistent Chroma for live retrieval-augmented generation, and a deterministic offline fallback for reproducible testing. The submitted dataset contains 12 entirely fictional patients and eight scenarios. Four live prompt versions were evaluated with **gpt-5.4-mini**. The final v4 hybrid design achieved 100% scenario pass rate, urgency accuracy, required-source recall, source traceability, and safety compliance, with a 0% structural unsupported-source proxy and 15.65-second mean latency. These results establish consistency on the designed scenarios, not clinical validity.

## 1. Problem and motivation

The three data sources central to remote chronic-care monitoring describe different parts of the same patient story. EHR data describes diagnosed conditions, medication orders, labs, and encounters. RPM captures current measurements and trends. Anamnesis describes symptoms, adherence, lifestyle, and patient concerns. When an alert is reviewed without the other two streams, the reader must either search manually or act on an incomplete picture.

ClinicalBridge treats this as a retrieval, language, and orchestration problem. The objective is not to automate diagnosis. It is to reduce the time needed to assemble traceable context while making uncertainty and disagreement visible.

## 2. Objective and scope

The implemented objective is to accept a simulated RPM alert and produce a structured brief that a clinician could read in under 60 seconds. The brief identifies urgency, summarizes the alert, presents only relevant history, correlates patient-reported context, lists non-diagnostic risk considerations, suggests review actions, and exposes missing information.

The project includes simulated records, specialized prompts, RAG, memory, tools, evaluation, a CLI, a Streamlit interface, and an annotated notebook. It excludes real patient data, production deployment, regulatory claims, device integration, and automated treatment decisions.

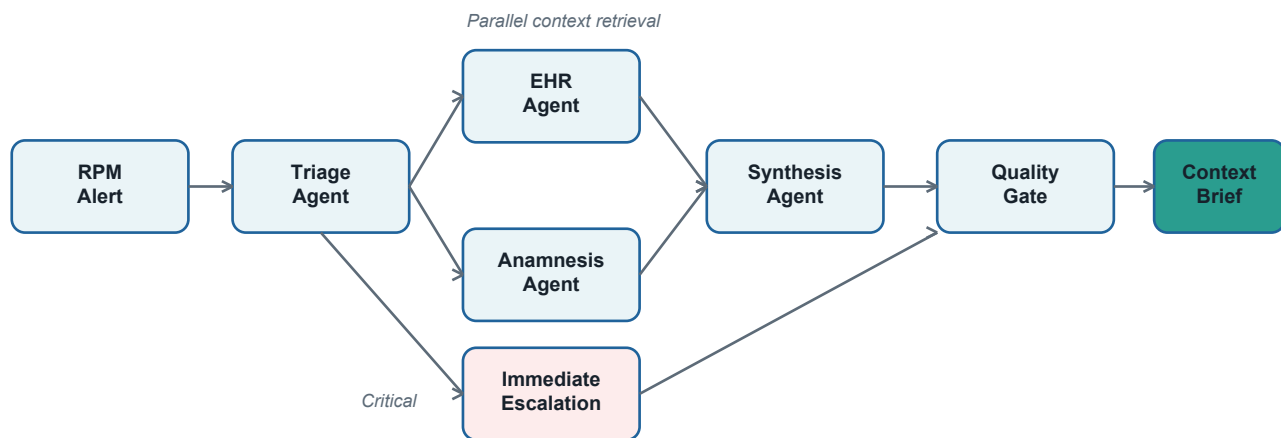
## 3. Requirements interpretation

The capstone specifies four agents and a central orchestrator. ClinicalBridge implements each component as a typed unit:

- **RPMAlert** is the entry contract.
- **TriageDecision** defines urgency and retrieval needs.
- **EHRContext** contains retrieved evidence and gaps.
- **AnamnesisSummary** contains patient-reported context.
- **ClinicalContextBrief** is the final product.

The brief's requirement for chain-of-thought design is implemented as explicit task decomposition, concise decision factors, and source-linked intermediate structures. The project does not request or store hidden model chain-of-thought.

## 4. System architecture



The Alert Triage Agent processes each alert first. Critical alerts are diverted immediately to the fail-safe escalation node. Non-critical alerts pass to a dispatch node that starts EHR retrieval and anamnesis interpretation in parallel. Their structured outputs converge at the Synthesis Agent. A quality gate checks source IDs, mandatory human review, and disclaimer presence.

The orchestrator owns coordination rather than clinical reasoning. This separation allows each prompt and metric to be improved independently.

## 5. OpenAI and LangChain implementation

The live model default is **gpt-5.4-mini**, selected as a cost-latency compromise for repeated structured calls. The model remains configurable through **.env**. LangChain's **ChatOpenAI** integration uses the Responses API path. Triage, anamnesis, and synthesis use native JSON-schema output. The heterogeneous EHR context uses OpenAI function calling because strict JSON Schema rejects arbitrary clinical dictionaries; Pydantic still validates the returned object. The EHR vector store uses **text-embedding-3-small** and persistent Chroma.

The direct OpenAI SDK was considered but not used as the sole application layer because the course requires LangChain pipelines and multi-agent orchestration. LangGraph expresses conditional safety routing and parallel context retrieval directly.

## 6. Retrieval-augmented generation

Each fictional EHR record is split into source-level chunks for demographics, conditions, medications, labs, notes, and allergies. Every chunk carries patient ID, section, original payload, and immutable source ID. Live retrieval filters on patient ID, preventing cross-patient results.

The local fallback uses TF-IDF with the same patient filter. It supports tests and demonstrations without external calls, but it is documented as a regression mechanism rather than a semantic-equivalent replacement for embeddings.

## 7. Memory and tools

Each workflow run creates event memory, summary memory, and entity memory. Event memory records agent events and structured payloads. Summary memory records compact workflow milestones. Entity memory stores patient-scoped facts such as urgency and patient concerns. Runtime logs are excluded from version control.

Tools include deterministic alert classification, EHR retrieval, source-level parsing, and Pydantic validation. Live evaluation showed that free-form urgency assignment remained unstable across prompt versions. In the final v4 design, the deterministic tool owns the urgency enum while the LLM explains the result and formulates retrieval questions.

## 8. Simulated dataset

The generated cohort contains 12 fictional patients covering common chronic conditions. Each patient has EHR and anamnesis records; the RPM CSV contains background observations and alert-triggering measurements. The manifest explicitly states that no real patient data is present.

The eight scenarios cover medication interruption, contextual false alarms, trend deterioration, incomplete records, conflicting data, critical hypoxemia, device artifact, and medication-related contextual factors. Each scenario defines expected urgency, key concerns, and required source IDs.

## 9. Prompt engineering

Each agent has three full prompt versions and three few-shot examples. Version 1 establishes a baseline. Version 2 introduces explicit process and structure. Version 3 adds safety boundaries, missing-data behavior, traceability, confidence calibration, contradiction handling, and negative constraints. A fourth triage-only iteration was added after live testing to establish deterministic tool authority for urgency.

The v3 EHR, anamnesis, synthesis, and triage prompts each satisfy all eight static prompt-quality checks. Static completeness alone did not predict live performance, so v4 addressed the architectural cause rather than expanding the prompt further.

## 10. Safety design

Safety is enforced in multiple layers rather than left to one disclaimer. The dataset is verified as simulated. Critical thresholds follow deterministic rules. Critical alerts bypass routine synthesis. Prompts prohibit diagnosis, prescription, and unsupported facts. Every substantive output item carries source IDs. The synthesis agent uses an upstream source allow-list, performs one corrective retry, and reconstructs citations from genuine source objects. The quality gate restores mandatory safety fields. Tests cover both critical and non-critical paths.

## 11. Evaluation

The final live v4 run processed all eight scenarios. Aggregate results were:

- scenario pass rate: 100%;
- urgency accuracy: 100%;
- required EHR source recall: 100%;
- required anamnesis source recall: 100%;
- source traceability: 100%;
- structural unsupported-claim proxy: 0%;
- safety compliance: 100%;
- mean key-concern lexical coverage: 76.32%;
- mean live latency: 15.65 seconds;
- maximum live latency: 20.16 seconds.

The live prompt comparison produced pass rates of 87.5% for v1, 62.5% for v2, 75% for v3, and 100% for v4. v2 demonstrated that a longer, more explicit prompt is not automatically better. v3 recovered the device-artifact and medication-discrepancy cases. v4 fixed the remaining urgency instability by assigning the deterministic triage tool authority over the urgency enum.

The live smoke tests also exposed two integration failures. OpenAI strict JSON Schema rejected heterogeneous EHR dictionaries, requiring function calling for that agent. The model also invented plausible source IDs, leading to the source allow-list and deterministic citation reconstruction.

Nine automated tests passed, including a Streamlit widget and scenario-execution test. Lint checks passed. The CLI doctor confirmed 12 patients, eight scenarios, the selected model, and the offline fallback.

## 12. Interfaces and demonstration

The CLI supports environment checks, scenario listing, scenario execution, custom alert files, JSON output, and Markdown export. The Streamlit dashboard presents a scenario, alert, gold evaluation target, generated brief, and expandable agent audit trail.

The annotated notebook demonstrates scenario 03 and scenario 06. The pair was selected because it contrasts the full parallel pipeline with the critical bypass pathway.

## 13. Limitations

The results are bounded to a small, authored dataset. The records do not reproduce real EHR messiness. The evaluation gold standards are not clinically validated. Structural citation checks cannot prove entailment. Confidence scores are not calibrated clinical probabilities. Multi-agent decomposition introduces cascading failure risk. The offline fallback does not predict live model behavior.

No claim is made that ClinicalBridge is safe or effective for real care. Production use would require clinical validation, regulatory review, security controls, privacy governance, human-factors testing, and substantially stronger evidence verification.

## 14. Reflection

The most important design lesson was that prompt quality cannot compensate for missing contracts. Source IDs, schemas, critical routing, and evaluation targets had to be defined before polished language became useful. The second lesson was that multi-agent systems create clarity only when responsibilities remain narrow. The orchestrator does not synthesize, retrieval agents do not diagnose, and the synthesis agent cannot silently invent evidence.

The evaluation failures were instructive. The system could be structurally grounded but semantically incomplete, and a statically stronger prompt could perform worse. Traceability, completeness, classification behavior, and orchestration guardrails must therefore be measured separately.

## 15. Conclusion

ClinicalBridge demonstrates prompt engineering as system design rather than chatbot phrasing. The project integrates role-specific prompts, structured outputs, RAG, tools, memory, orchestration, safety guardrails, and evaluation into one coherent proof of concept. Its strongest result is not a clinical claim; it is a transparent implementation in which every designed scenario can be reproduced, audited, and improved.

## References

### Clinical context and health informatics

- Wright, A., et al. (2015). Problem list completeness in electronic health records: A multi-site study and assessment of success factors. *International Journal of Medical Informatics*.
- Hravnak, M., et al. (2018). Cardiorespiratory instability before and after implementing an integrated monitoring system. *Critical Care Medicine*.
- Senbekov, M., et al. (2020). The recent progress and applications of digital technologies in healthcare: A review. *International Journal of Environmental Research and Public Health*.
- Adler-Milstein, J., & Jha, A. K. (2017). HITECH Act drove large gains in hospital electronic health record adoption. *Health Affairs*.

### Remote patient monitoring

- Vegesna, A., et al. (2017). Remote patient monitoring via non-invasive digital technologies: A systematic review. *Telemedicine and e-Health*.
- Cvach, M. (2012). Monitor alarm fatigue: An integrative review. *Biomedical Instrumentation & Technology*.
- Noah, B., et al. (2018). Impact of remote patient monitoring on clinical outcomes: An updated meta-analysis of randomized controlled trials. *NPJ Digital Medicine*.

### Anamnesis and patient-reported data

- Bates, D. W., et al. (2014). Big data in health care: Using analytics to identify and manage high-risk and high-cost patients. *Health Affairs*.
- Bachmann, M., & Windak, A. (2020). Digital anamnesis in primary care: Potential and limitations. *Healthcare*.

### LLMs, prompting, and agents

- Singhal, K., et al. (2023). Large language models encode clinical knowledge. *Nature*.
- Thirunavukarasu, A. J., et al. (2023). Large language models in medicine. *Nature Medicine*.
- Wei, J., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*.
- Yao, S., et al. (2023). ReAct: Synergizing reasoning and acting in language models. *ICLR*.
- Park, J. S., et al. (2023). Generative agents: Interactive simulacra of human behavior. *UIST*.

### Current implementation documentation

- [OpenAI Models](#)
- [OpenAI Structured Outputs](#)
- [OpenAI Embeddings](#)
- [LangChain ChatOpenAI](#)
- [LangChain Chroma](#)
- [LangGraph Workflows and Agents](#)